DEVSECOPS LAB BOOK · PHASE 4

Supply Chain Security & Compliance

Chapters 18–19 | SLSA · Attestation · Signing · SOC2 · PCI-DSS · HIPAA

SLSA LEVELS	FRAMEWORKS	GOAL
1–4	SOC2 · PCI · HIPAA	Complete Traceability

Chapter Goal

Implement SLSA (Supply-chain Levels for Software Artifacts). Ensure artifacts are properly built, signed, and attested throughout the pipeline.

- 18.1 Image Signing & Verification
- 18.2 Build Provenance & Attestation
- 18.3 Supply Chain Levels (SLSA)
- 18.4 Dependency Management (SBOM)
- 18.5 Vulnerability Attestation

SLSA LEVELS

1 Scripted builds
COMPLETE

2 Version control + CI/CD
IN PROGRESS

3 Signed provenance + hardened CI/CD
FUTURE

4 Fully hermetic builds
FUTURE

Docker Content Trust & Harbor Signing



Sign every image before it leaves the pipeline; verify on every pull.

GITLAB CI/CD

```
export DOCKER_CONTENT_TRUST=1
export DOCKER_CONTENT_TRUST_SERVER=\
  https://notary.docker.io

docker build -t $IMAGE_NAME:$CI_COMMIT_SHA .
docker push $IMAGE_NAME:$CI_COMMIT_SHA
  # prompts for delegation key passphrase

docker pull $IMAGE_NAME:$CI_COMMIT_SHA
  # client verifies signature on pull
```

HARBOR REGISTRY POLICY

Enable content trust

Per-project: Projects → myapp → Enable content trust

Require signature before push

Administration → System Settings

Block unsigned images

Reject any image without a valid signature

Generate a Signed Attestation for Every Build



1

Build

Compute image digest via docker inspect



2

Assemble

Populate provenance.json — builder, materials, metadata



3

Sign

gpg --armor --detach-sign provenance.json



4

Publish

POST provenance + signature to attestation store

KEY FIELDS IN provenance.json

`artifact.uri / digest`	Image reference + content digest	`builder.id`	https://gitlab.devsecops.com
`buildConfig.source`	$CI_PROJECT_URL + entrypoint (.gitlab-ci.yml)	`metadata.buildInvocationId`	$CI_PIPELINE_ID, build start time
`metadata.completeness`	parameters / environment / materials flags	`materials[]`	Source commit URI + SHA

Tracking SLSA Compliance

1

Scripted Builds

✓ COMPLETE

- Scripted builds
- Build output retention
- Build process documentation

2

Version Control + CI/CD

▸ IMPLEMENTING

- Version controlled source
- Tracked, automated builds
- Cryptographic provenance
- Access control

3

Hardened CI/CD

FUTURE

- Signed, reviewed commits
- Hardened CI/CD
- Verified dependencies
- Hermetic builds

4

Fully Hermetic

FUTURE

- Fully hermetic builds
- Bit-for-bit reproducible
- Offline signing
- Auditability

Software Bill of Materials (SBOM)

Track every dependency inside a container image with syft.

GENERATE SBOM

```
syft $IMAGE_NAME:$CI_COMMIT_SHA -o json > sbom.json
syft $IMAGE_NAME:$CI_COMMIT_SHA -o spdx > sbom.spdx
```



Base OS packages

e.g. openssl 1.1.1



Language packages

e.g. Flask, requests



System libraries

e.g. libc, libssl

EXAMPLE SBOM ENTRY

```
{
  "name": "openssl",
  "version": "1.1.1q",
  "type": "rpm",
  "purl": "pkg:rpm/openssl@1.1.1q?arch=x86_64",
  "licenses": ["OpenSSL"]
}
```

SBOM formats produced

- CycloneDX (JSON)
- SPDX (tag-value & JSON)

Attesting That an Image Has Been Scanned

ATTESTATION RECORD

```
{
  "type": "vulnerability-scan",
  "timestamp": "2025-01-15T14:30:00Z",
  "image": "harbor.devsecops.com/myapp/myapp:v1.0",
  "scanner": "Trivy",
  "scanResult": {
    "critical": 0,
    "high": 0,
    "medium": 2,
    "low": 5
  },
  "passed": true,
  "signed": true,
  "signature": "-----BEGIN PGP SIGNATURE-------..."
}
```

0	CRITICAL
0	HIGH
2	MEDIUM
5	LOW



Scan passed — image cleared for signing & release

Chapter Goal

Implement compliance requirements (SOC2, PCI-DSS, HIPAA) through automated evidence collection and reporting.

- 19.1 Compliance Frameworks (SOC2 / PCI-DSS)
- 19.2 Automated Compliance Evidence Collection
- 19.3 SBOM Generation & Distribution
- 19.4 Vulnerability Disclosure & SLA
- 19.5 Compliance Reporting
- 19.6 Continuous Compliance Monitoring
- 19.7 Incident Response Integration

SOC2

Type II

PCI-DSS

Req. 6 & 8

HIPAA

Safeguards

Mapping DevSecOps Controls to Requirements

SOC2 TYPE II

CC7.1 Restrict System Access

- ✓ OIDC authentication (Authentik)
- ✓ RBAC in all tools
- ✓ Session recording (Teleport)
- ✓ Audit logs (all phases)

CC7.2 Software Security

- ✓ Code review (GitLab MRs)
- ✓ SAST scanning (SonarQube)
- ✓ Container scanning (Harbor/Trivy)
- ✓ Dependency scanning

CC9.2 Configuration

- ✓ Infrastructure as code (OpenTofu)
- ✓ Version controlled configs
- ✓ Change auditing

PCI-DSS

Req. 6 Secure Development

- ✓ Code review process
- ✓ Static testing (SonarQube)
- ✓ Vulnerability remediation tracking (DefectDojo)

Req. 8 User Access

- ✓ Authentication (Authentik OIDC)
- ✓ MFA (via Authentik)
- ✓ Access logging (Teleport)

Daily Compliance Evidence Export

Scheduled job pulls evidence from every tool in the pipeline and packages it for auditors.



GitLab

Merge requests — code review evidence



SonarQube

Code quality metrics & issues



Harbor

Image scan results



Vault

Secrets audit log



Teleport

Session recordings & audit

EXPORT PIPELINE

```
Collect from all sources  →  Write to /var/reports/  →  tar -czf compliance-DATE.tgz  →  Cron: 0 2 * * *
```

Build-Time SBOM: Generate, Sign, Publish

1



Build image

```
docker build -t $IMAGE:$TAG .
```

2



Generate SBOM

```
syft → CycloneDX + SPDX-JSON
```

3



Sign SBOM

```
gpg --armor --detach-sign sbom.xml
```

4



Attach annotation

```
crane mutate --annotation sbom=@...
```

5



Publish

```
POST to sbom-repo.devsecops.com
```



SBOM published and auditable — auditors can verify exactly what's in each image.

Formats: CycloneDX XML · SPDX JSON Distribution: sbom-repo.devsecops.com/api/upload

Remediation SLA Policy

Tracked automatically in DefectDojo — discovery date, due date, status, days remaining.

CRITICAL	HIGH	MEDIUM	LOW
24 hours	7 days	30 days	90 days
Fix or disable	Fix required	Fix required	Fix required

ESCALATION PATH



7 days before SLA

Notify team lead



At SLA

Escalate to manager



7 days overdue

Executive escalation

Annual Compliance Report — Sample Metrics

Period: 2025-01-01 to 2025-12-31

100%

OIDC auth coverage

95%

Code review rate
(4,750 / 5,000 commits)

100%

SAST + container
scan coverage

100%

Critical findings
fixed on SLA

VULNERABILITY METRICS

- ✓ Total findings: 245
- ✓ Critical fixed: 8/8 (100%)
- ✓ High fixed: 42/43 (98%)
- ✓ Overdue: 0
- ✓ Compliance: 100%

AUDIT TRAIL

- ✓ Total events logged: 2.3M
- ✓ Retention period: 1 year
- ✓ Search capability: Real-time

Certified by CISO — signed 2026-01-15, per SOC2 / PCI-DSS / HIPAA requirements.

Prometheus / Grafana Compliance Alerts

CRITICAL	Critical vulnerabilities overdue	`count(critical AND days_overdue > 0) > 0`
HIGH	High vulnerabilities SLA breach	`count(high AND days_overdue > 7) > 0`
MEDIUM	Code review coverage	`reviewed / total < 0.90`
CRITICAL	SAST scan coverage	`commits_scanned / total < 1.0`
CRITICAL	Image scan coverage	`images_scanned / pushed < 1.0`

Reconstructing an Incident from Audit Logs

Example: unauthorized access attempt at 2025-01-15 14:30:00

Teleport

Session logs — filter failed user_login events

Vault

Audit log — cross-reference source IP

HAProxy

Access logs — grep by IP address

RECONSTRUCTED EVENT

```
{
  "timestamp": "2025-01-15T14:30:45Z",
  "event": "user_login",
  "user": "unknown",
  "ip": "203.0.113.45",
  "reason": "Invalid credentials",
  "status": "failed"
}
```

Complete incident timeline reconstructed from the audit trail.

CHAPTER SUMMARY

Complete Traceability & Compliance

CH.18 **SLSA Framework —** Image signing, provenance attestation

CH.19 **Compliance —** SBOM, SLA tracking, audit reports

KEY TAKEAWAY — EVERY ARTIFACT HAS:



Attestation

Who built it, when, how



SBOM

What's in it, dependencies



Signature

Cryptographically verified



Audit trail

All operations logged



Compliance mapping

SOC2 / PCI / HIPAA ready